Author: Luiz Henrique Mendonça Nr. 17442671

Supervisor: Prof. Dr.-Ing. Stötzer

Scientific Report: Digital Guitar Effect Pedal with a Delay Effect on an STM32H743

Hochschule Ravensburg-Weingarten

University of Applied Sciences

Contents

1 Abstract

This report describes the design and build of a digital guitar effect pedal that adds an echo (delay) to the signal of an electric guitar. The pedal sits in the signal chain between the guitar and the amplifier, and is built around an STM32H743 microcontroller with a minimal analog frontend and backend based on a single MCP6002 dual op-amp running on a $3.3\,\mathrm{V}$ single supply. The frontend conditions the weak guitar signal so that it can be sampled by the on-chip ADC. The firmware stores each sample in a circular buffer and plays it back $300\,\mathrm{ms}$ later mixed with the live signal. The backend brings the processed output back to a level and impedance that a guitar amplifier accepts. The full chain was designed, simulated, prototyped on a breadboard, soldered onto a protoboard, and finally mounted in a 3D-printed enclosure. The prototype produced a clearly audible echo on real guitar input, fulfilling the proposed goals.

2 Introduction

A delay pedal stores what the guitar plays and repeats it a short time later. The listener hears the original note and then a quieter copy of it; if part of the copy is fed back into the delay line, the echoes repeat and gradually fade. The effect has been used in guitar music since the tape-echo devices of the 1950s, and today it can be reproduced with a microcontroller and a handful of passive components. It is a good showcase for a digital effect pedal precisely because long, clean delays are difficult to achieve in purely analog hardware.

Compared to dedicated audio codecs or application-specific DSP chips, this minimal-BOM approach leverages the high clock speed and built-in converters of the STM32H7, alongside a standard single-supply op-amp. This design trades audio fidelity for hardware simplicity, providing a practical context to explore mixed-signal embedded engineering constraints.

The central technical problem is moving the analog signal in and out of the digital domain without introducing noise. A guitar pickup produces a continuous voltage that swings symmetrically around ground, while a microcontroller works with discrete numbers sampled at a fixed rate. On top of that, the Nyquist theorem requires that no frequency component above half the sampling rate reaches the converter; otherwise that energy folds back into the audible band as aliasing and produces tones that were never in the original signal. To prevent this, a low-pass filter is placed before the ADC to remove out-of-band content before sampling, and an identical filter is placed after the DAC to smooth the stepped output back into a continuous signal. At a sampling rate of $48\,\mathrm{kHz}$ the Nyquist limit limit is $24\,\mathrm{kHz}$, which covers the full audible range. In practice the energy of music sits well below this limit. Figure 1 shows the frequency distribution of a typical music signal: most of the energy is below $5\,\mathrm{kHz}$ and the audible range tapers off well before $20\,\mathrm{kHz}$. The anti-alias and reconstruction filters in this project were sized

with this in mind.

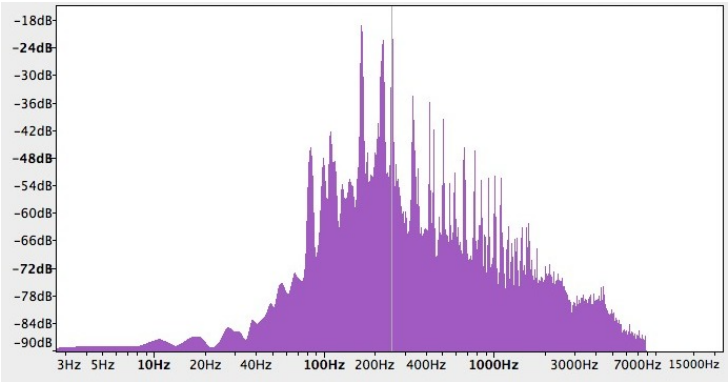

Figure 1: Frequency distribution of a typical music signal. Most energy is below $5\,\mathrm{kHz}$ and the audible range tapers off well before $20\,\mathrm{kHz}$.

The analog signal path uses an op-amp as a unity-gain buffer at the input and at the output, to give the filter stages a high input impedance toward the guitar pickup and a low output impedance toward the next stage, so the cutoff of each RC filter is not pulled by the connected load. A high-pass filter at the output removes the DC bias that the single-supply circuit adds to the signal, before it reaches the downstream amplifier. At the ADC input, a pair of BAT54S Schottky diodes clamps the pin to the supply rails to protect the converter from over-voltage.

On the digital side, a hardware timer drives the sample rate by firing an update event at $48\,\mathrm{kHz}$, which starts an ADC conversion and latches a DAC output in hardware without involving the processor. DMA moves data between the converter peripherals and memory buffers, so the CPU is free to run the delay algorithm while transfers proceed in the background. Double buffering divides the DMA buffer into two halves: the hardware fills one half while the processor works on the other, keeping the audio stream continuous with no gap between conversions.

The goal of this project was to build a working delay pedal at $48\,\mathrm{kHz}$ with up to $300\,\mathrm{ms}$ of delay, using a single MCP6002 [2] dual op-amp for both analog stages

on a $3.3\,\mathrm{V}$ supply, and an STM32H743 microcontroller [1] for the digital signal processing. The project question follows from the goal: *Can a working digital delay effect for an electric guitar be built around an STM32H743 and a single MCP6002 dual op-amp on a* $3.3\,\mathrm{V}$ *single supply, producing an audible echo on real guitar input?*

3 Materials and Methods

3.1 Materials Used

The microcontroller used was an STM32H743VIT6 [1] on a WeAct Mini development board (Figure 2). It is an ARM Cortex-M7 running at up to $480\,\mathrm{MHz}$, with a 16-bit ADC, a 12-bit DAC, and a DMA controller that moves data between peripherals and memory without CPU involvement. The analog stages were built around one MCP6002 dual op-amp and a small set of resistors and capacitors. Two $6.35\,\mathrm{mm}$ TS jack jacks carry the input and output signals. All the components and their values are documented in the circuit diagrams. A Bill of Material (BoM) can also be provided as an attachment.

Figure 2: WeAct Mini STM32H743 development board.

The following tools were used during the project:

- STM32CubeMX, to generate the peripheral initialization code.
- `arm-none-eabi-gcc` version 12 as cross-compiler, called from CMake.
- KiCad 8, for the schematic capture.

- Falstad Circuit Simulator, to check the full analog signal path in the browser before building it.
- A guitar, a guitar amplifier, and an oscilloscope, for listening tests and waveform verification.
- Audacity 3, for recording and analysing the pedal output during the verification measurements.

3.2 Methods

A top-down approach was taken for developing the system architecture. The sampling rate was fixed first at $48\,\mathrm{kHz}$, which set the Nyquist limit and from there the cutoff of the anti-alias filter. $48\,\mathrm{kHz}$ was chosen over the $44.1\,\mathrm{kHz}$ CD rate because it is the standard rate of professional digital audio equipment and divides evenly into the $240\,\mathrm{MHz}$ timer clock, which avoids drift between the timer period and the actual sample rate. The component values were then calculated based on that and the input range the ADC needs based on the STM32 datasheet (so we know which input and output are expected from the ADC and DAC). Each analog stage was built on a breadboard and tested on its own with a signal generator before being connected to the microcontroller, so that problems could be isolated to one part of the chain at a time.

The firmware was developed in parallel with the hardware. STM32CubeMX was used to generate the initialization code for the ADC, DAC, timer, and DMA peripherals; the DSP loop was then developed in a separate module (dsp.h) and integrated into the DMA callbacks. The code was compiled with CMake and flashed over ST-Link. The waveforms were verified on the oscilloscope using a simulated signal from a frequency generator, which made it possible to test the firmware on its own without depending on the analog stages. The circuit was then

moved from the breadboard to a hand-soldered protoboard and assembled in a 3D-printed enclosure (Figure 3).

The verification measurements (results documented in Section 4.6) used the setup: A $300\,\mathrm{Hz}$ sine wave generated on the laptop was sent from the laptop headphone output into the pedal input jack J1, processed by the pedal, fed into the guitar amplifier, and recorded back into Audacity through the amplifier's USB output at $48\,\mathrm{kHz}$, 16-bit. A second recording was taken with the laptop disconnected from J1, capturing the noise alone. Both recordings used identical amplifier and recorder gain settings, so the gain factor cancels in the ratio of the two and the system SNR can be derived directly from the dBFS difference between them.

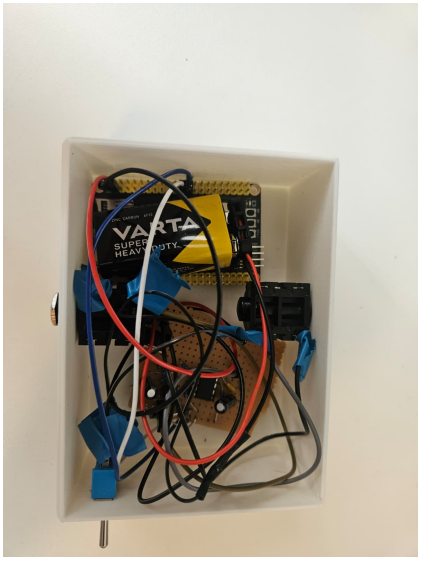

Figure 3: Final prototype in its 3D-printed enclosure, powered by a $9\,\mathrm{V}$ battery.

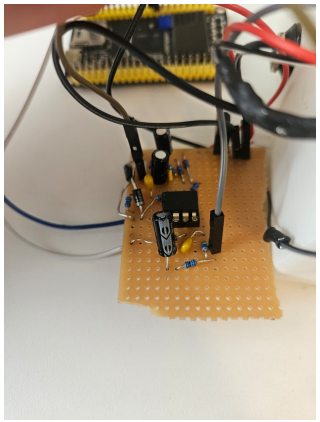

Figure 4: Hand-soldered protoboard with the analog frontend, backend, and STM32H743 development board.

4 Results

The results is the design of a system that was divided in two main parts: the analog frontend/backend, and the software architecture of the firmware. In the sections below both designs are described and explained:

4.1 Analog Frontend

The frontend conditions the guitar signal before it reaches the ADC. Figure 5 shows the schematic. Signal flow is left to right; each component is described below.

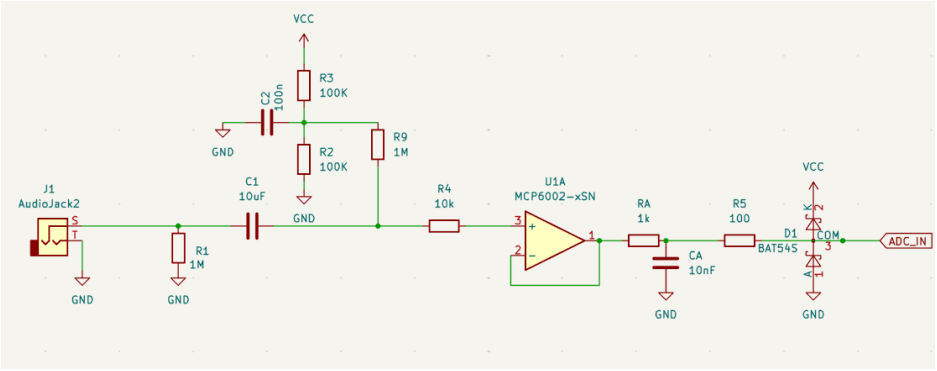

Figure 5: Analog frontend: guitar input to ADC pin.

J1 — 6.35 mm TS jack. Guitar input.

R1 — $1\,\mathrm{M\Omega}$ **to GND.** Sets input impedance to $1\,\mathrm{M\Omega}$, which is what a guitar pickup expects. Also bleeds charge off C1 when the cable is unplugged, preventing a pop on reconnection.

C1 — $10\,\mu\mathrm{F}$**.** AC-coupling capacitor. Blocks any DC from the source; passes audio. The high-pass corner with R1 is

$$f_c = \frac{1}{2\pi \cdot 1\,\mathrm{M\Omega} \cdot 10\,\mu\mathrm{F}} \approx 0.016\,\mathrm{Hz}, \tag{4.1}$$

well below the lowest guitar note, so no signal is lost.

R2, R3 — $100\,\mathrm{k}\Omega$ **each (bias divider).** Divide the $3.3\,\mathrm{V}$ supply to produce a DC midpoint:

$$V_{\text{bias}} = \frac{3.3}{2} = 1.65\,\mathrm{V}. \tag{4.2}$$

The guitar signal swings around $0\,\mathrm{V}$; the ADC input range is $0\,\mathrm{V}$ to $3.3\,\mathrm{V}$. Lifting the signal onto the $1.65\,\mathrm{V}$ midpoint centers its swing inside the full ADC range.

C2 — $100\,\mathrm{nF}$**.** Decouples the bias node to GND. Keeps the $1.65\,\mathrm{V}$ reference stable against VCC variations.

R9 — $1\,\mathrm{M}\Omega$**.** Connects the bias node to the signal path. The Thévenin impedance of R2/R3 is $50\,\mathrm{k}\Omega$. Without R9, that $50\,\mathrm{k}\Omega$ would load the signal and interact with the source impedance, attenuating high frequencies (tone suck). R9 raises the impedance seen by the signal to $50\,\mathrm{k}\Omega + 1\,\mathrm{M}\Omega \approx 1.05\,\mathrm{M}\Omega$, which is negligible next to R1.

R4 — $10\,\mathrm{k}\Omega$**.** Series resistor at the op-amp non-inverting input. Limits input current in fault conditions and damps any resonance with op-amp input capacitance.

U1A — MCP6002, unity-gain buffer. Output tied directly to inverting input. Input impedance $\approx 10^{12}\,\Omega$; output impedance $< 1\,\Omega$. Isolates the high-impedance bias divider from the low-impedance filter that follows.

RA, CA — $1\,\mathrm{k}\Omega$ **and** $10\,\mathrm{nF}$ **C0G.** Anti-aliasing low-pass filter:

$$f_c = \frac{1}{2\pi \cdot 1\,\mathrm{k}\Omega \cdot 10\,\mathrm{nF}} \approx 15.9\,\mathrm{kHz}. \tag{4.3}$$

The cutoff sits above the band where musical content lives (Figure 1) and below the $24\,\mathrm{kHz}$ Nyquist limit, so it removes out-of-band energy before sampling without cutting useful signal.

R5 — $100\,\Omega$**.** Protection resistor at the ADC pin. Limits fault current through the STM32 internal ESD diodes if a transient drives the pin outside the supply rails.

D1 — BAT54S dual Schottky. Clamp diodes at the ADC pin. One diode from

pin to VCC, one from pin to GND. Forward voltage $\approx 0.3\,\mathrm{V}$: the pin is clamped to the range $[-0.3\,\mathrm{V},\ 3.6\,\mathrm{V}]$ before any transient reaches the converter input.

4.2 Analog Backend

The backend converts the DAC output back to a line-level audio signal. Figure 6 shows the schematic. Signal flow is left to right; each component is described below.

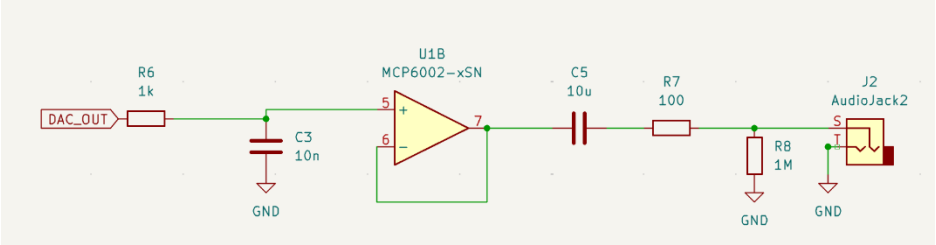

Figure 6: Analog backend: DAC output to amplifier jack.

R6 — $1\,\mathrm{k}\Omega$. Series resistor at the DAC output. Together with C3 it forms the reconstruction low-pass filter (see C3 below).

C3 — $10\,\mathrm{nF}$ C0G. Together with R6, forms the reconstruction low-pass filter:

$$f_c = \frac{1}{2\pi \cdot 1\,\mathrm{k}\Omega \cdot 10\,\mathrm{nF}} \approx 15.9\,\mathrm{kHz}. \tag{4.4}$$

The DAC holds each sample constant until the next clock tick, producing a staircase waveform. The filter smooths those steps before the signal reaches the op-amp. The cutoff matches the anti-aliasing filter on the frontend, so both edges of the digital window are treated symmetrically.

U1B — MCP6002, unity-gain buffer. Second channel of the same dual op-amp used in the frontend. Output tied directly to inverting input. Presents a high-impedance load to C3 so the filter corner stays at $15.9\,\mathrm{kHz}$, and drives the output network from a low-impedance source.

C5 — $10\,\mu\mathrm{F}$**.** AC-coupling capacitor at the op-amp output. The DAC runs with a $1.65\,\mathrm{V}$ DC midpoint bias; C5 blocks this offset so the output delivered to the amplifier is centered on $0\,\mathrm{V}$.

R7 — $100\,\Omega$**.** Series resistor at the output node. Limits fault current through the op-amp output stage if the jack is shorted or connected to a low-impedance load, and damps any cable capacitance resonance.

R8 — $1\,\mathrm{M}\Omega$ **to GND.** Bleeder resistor at the output node. Provides a DC return path so C5 does not float the output when no amplifier is connected. Also forms a high-pass corner with C5:

$$f_c = \frac{1}{2\pi \cdot 1\,\mathrm{M}\Omega \cdot 10\,\mu\mathrm{F}} \approx 0.016\,\mathrm{Hz}, \tag{4.5}$$

negligible relative to the audio band.

J2 — 6.35 mm TS jack. Output to guitar amplifier.

The full analog path was also entered into the Falstad simulator to check the frontend and the backend together before building the circuit. Figure 7 shows the simulated chain.

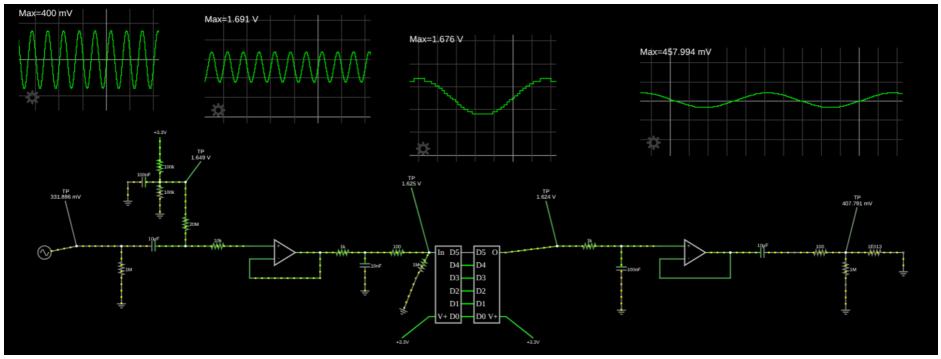

Figure 7: Falstad simulation of the full analog path: frontend on the left, backend on the right. Simulated ADC-DAC in the middle

4.3 Software Architecture

The firmware is built around the idea that the CPU should not be involved in the timing of the signal chain at all. The sampling rate is generated in hardware, the data is moved by DMA, and the CPU only wakes up when a new block of samples is ready to process. Figure 8 shows the overall flow.

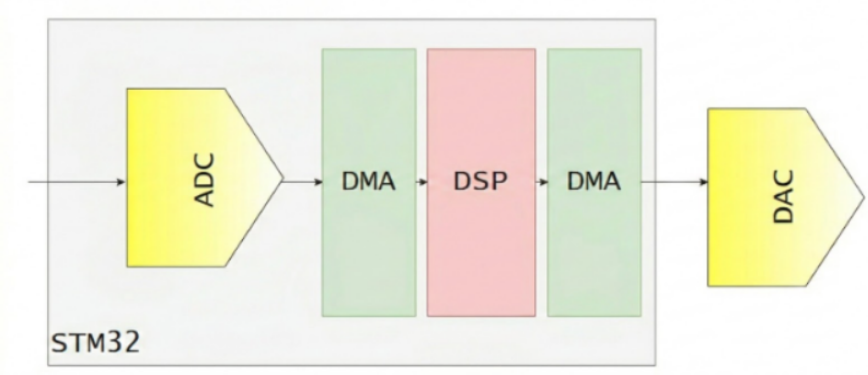

Figure 8: Signal flow from guitar input to amplifier output.

The internal HSI $64\,\mathrm{MHz}$ RC oscillator is configured through the PLL to produce a $480\,\mathrm{MHz}$ system clock. After the bus prescalers, TIM6 receives a $240\,\mathrm{MHz}$ input clock. With a counter period of 5000 ticks, it fires an update event at

$$f_s = \frac{240\,\mathrm{MHz}}{5000} = 48\,\mathrm{kHz}. \tag{4.6}$$

Listing 1 shows the relevant lines from `Src/tim.c` (file auto-generated by STM32CubeMX HAL boilerplate generator). The period register is written as `5000 - 1` because the counter counts from zero to `Period` inclusive, so 5000 ticks elapse before each reset. The `MasterOutputTrigger` setting routes that reset event onto the internal trigger bus (TRGO) so ADC1 and DAC1 can act on it without any software involvement.

```
1 htim6.Instance               = TIM6;
2 htim6.Init.Prescaler         = 0;
3 htim6.Init.CounterMode       = TIM_COUNTERMODE_UP;
4 htim6.Init.Period            = 5000 - 1;   // 240 MHz / 5000 =
    48 kHz
5 htim6.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
```

```
HAL_TIM_Base_Init(&htim6);

sMasterConfig.MasterOutputTrigger = TIM_TRGO_UPDATE;
HAL_TIMEx_MasterConfigSynchronization(&htim6, &sMasterConfig);
```

Listing 1: TIM6 period and master trigger configuration (`Src/tim.c`).

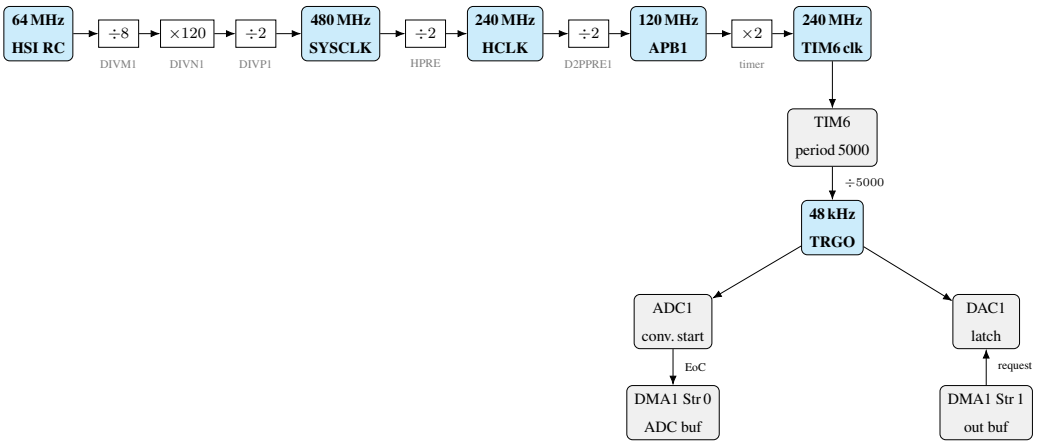

Figure 9: Clock path from HSI to the TIM6 TRGO pulse at $48\,\mathrm{kHz}$, and the resulting trigger chain to ADC1 and DAC1. Value boxes (blue) show the frequency at each node; prescaler boxes (white) show the division or multiplication applied. The $\times 2$ timer doubler is automatic on the STM32H7 whenever the APB1 prescaler is greater than 1, restoring the timer input to $240\,\mathrm{MHz}$. TIM6 counts 5000 ticks of its $240\,\mathrm{MHz}$ input and then resets, producing the $48\,\mathrm{kHz}$ update event. That event is broadcast on the internal trigger bus (TRGO) and starts one ADC conversion and latches one DAC output simultaneously. The DMA streams are then driven by the peripherals themselves (ADC end-of-conversion for DMA1 Stream 0; DAC sample-needed request for DMA1 Stream 1), not by TIM6 directly.

This event is routed to the internal trigger bus and used as the master tick for both the ADC and the DAC. Both peripherals are configured to act on that trigger: the ADC starts a conversion, the DAC latches the next output sample. Neither the

CPU nor any software is involved in keeping this timing; the hardware runs the loop on its own [5].

The DMA controller moves the data. The ADC fills a 256-sample circular buffer in memory, and the DAC drains a matching 256-sample output buffer. The most relevant code logic lies in how the CPU interacts with those buffers. The DMA raises an interrupt after the first 128 samples have been written (half-complete) and another after all 256 (complete). On the first interrupt, the CPU processes samples 0–127 of the input and writes results to samples 0–127 of the output. While it does that, the DMA is already filling samples 128–255 on the input side. On the second interrupt the roles swap: the CPU handles the second half while the DMA wraps back to the beginning. The CPU and the DMA always work on different halves, so neither ever has to wait for the other [4]. The DSP routine has at most

$$\frac{128}{48\,000\,\mathrm{Hz}} \approx 2.67\,\mathrm{ms} \tag{4.7}$$

to finish processing one half-buffer before the DMA overwrites it.

4.4 Delay Algorithm

The delay effect uses a circular buffer of 14 400 `int16_t` values. A write pointer advances one position per sample, and a read pointer follows 14 400 positions behind it; both wrap back to zero at the end of the buffer. Because the buffer is

$$\frac{14400}{48\,000\,\mathrm{Hz}} = 300\,\mathrm{ms} \tag{4.8}$$

long, the read pointer always returns a sample recorded $300\,\mathrm{ms}$ ago. No memory is ever copied or shifted; both pointers just step forward by one and wrap, so the cost per sample is independent of the delay length.

The DSP routine processes 128 samples per call, matching the half-buffer size. Listing 2 shows the complete loop.

```
#define DELAY_LEN 14400
static int16_t  delay_buf[DELAY_LEN];
static uint32_t write_idx = 0;
static uint32_t read_idx  = 1;

for (uint32_t i = 0; i < HALF_BUFFER; i++) {
    int32_t dry     = ((int32_t)input[i] - ADC_MID) >> 4;
    int32_t delayed = delay_buf[read_idx];

    int32_t fb = dry + (delayed >> 1);          // 50% feedback
    if (fb >  2047) fb =  2047;
    if (fb < -2048) fb = -2048;
    delay_buf[write_idx] = (int16_t)fb;

    int32_t out = DAC_MID + dry + ((delayed * 6) / 10); // 60%
   wet
    if (out > 4095) out = 4095;
    if (out < 0)    out = 0;
    output[i] = (uint16_t)out;

    write_idx = (write_idx + 1) % DELAY_LEN;
    read_idx  = (read_idx  + 1) % DELAY_LEN;
}
```

Listing 2: Delay DSP loop.

The first line inside the loop re-centers the ADC sample and scales it to fit the DAC range. The ADC produces unsigned 16-bit values (0 to 65535) centered at 32768 (`ADC_MID`). Subtracting the midpoint gives a signed value around zero; shifting right by four bits divides by sixteen, mapping the result to the range -2048 to $+2047$. This is the signed 12-bit range that the DAC works in. The shift costs

no multiplication operation and maintains the precision.

The next line reads `delayed`, which is whatever value was written into the buffer 14 400 samples ($300\,\mathrm{ms}$) ago. The feedback calculation that follows is the mechanism behind repeating echoes: half of that delayed value (`delayed » 1`, integer division by two) is added to the dry sample, and the result is written back into the buffer at `write_idx`. When this feedback value is read back $300\,\mathrm{ms}$ later, it will itself contain a mix of dry and previous echo, producing a new echo from an echo. Because the delayed component is halved on every pass, the echoes decay as $0.5,\ 0.25,\ 0.125, \ldots$ a geometric series with ratio 0.5 [3]. Reaching the conventional $-60\,\mathrm{dB}$ perceptual floor (amplitude 10^{-3} of the original) takes

$$n = \left\lceil \log_{0.5}(10^{-3}) \right\rceil = 10 \text{ repeats}, \tag{4.9}$$

which at $300\,\mathrm{ms}$ per repeat is about $3\,\mathrm{s}$ of audible tail before the echo dies out. The clamp to ± 2047 prevents integer overflow from accumulating; without it, a loud note could drive the feedback into saturation and the echoes would never fade.

The output line assembles the final sample. The dry signal is kept at full amplitude and the delayed signal is mixed in at 60 % (`(delayed * 6) / 10`), so the echo is clearly audible but quieter than the original note. `DAC_MID` (2048) is then added back to restore the unsigned offset that the 12-bit DAC requires. A second clamp keeps the result inside the 0–4095 range. Both index pointers advance and wrap at the end of each iteration. It should be noted that repeated integer division and truncation in this feedback loop introduces fresh quantization noise on each pass.

The double-buffering scheme adds one full buffer of pipeline latency:

$$t_{\text{lat}} = \frac{256}{48\,000\,\mathrm{Hz}} \approx 5.33\,\mathrm{ms}. \tag{4.10}$$

This is the unavoidable cost of processing samples in blocks rather than one at a time. Whether this delay is perceptible in playing is checked in Section 4.6.

4.5 Development Issues and Fixes

There were two main issues during development. First, the DMA1 controller cannot read the DTCM memory that the compiler uses by default. The audio buffers had to be set explicitly in the D2 SRAM region using a linker attribute. Before this fix, the system produced no sound and showed no error flags.

Second, a scaling bug showed how the algorithm depends on converter ranges. An early firmware version missed the right-shift on the first line of the DSP loop (Listing 2). Because of this, the raw 16-bit ADC sample (0–65535) was written directly to the 12-bit DAC buffer. Any sample value over 4095 saturated the DAC immediately, clipping the output to the maximum level for almost the whole waveform.

Figure 10 shows the three signals captured on the oscilloscope at that point. The yellow trace is the reference signal from a bench signal generator feeding the pedal input. The blue trace is the analog frontend output: the same signal shifted up by $1.65\,\mathrm{V}$ so that it sits in the positive range the ADC expects. The purple trace is the DAC output without the bit-shift correction: it is driven hard to the supply rail for most of each cycle and bears no resemblance to the input.

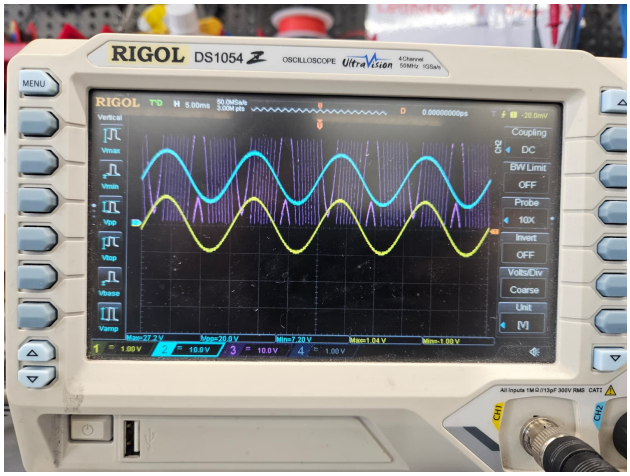

Figure 10: Oscilloscope capture during development. Yellow: signal generator input. Blue: analog frontend output (DC-biased to $1.65\,\mathrm{V}$). Purple: DAC output without the 16-bit-to-12-bit scaling, showing full-scale saturation.

Once the right-shift was added, the output waveform tracked the input correctly.

The bit-shift also sets the resolution of the reconstructed audio. The output dynamic range depends on the 12-bit DAC, not the 16-bit ADC, because the lower four bits of each sample are discarded.

In digital audio, every bit of resolution halves the quantization error, which adds roughly $6\,\mathrm{dB}$ to the signal-to-noise ratio. For a full-scale sine wave, the theoretical maximum signal-to-quantization-noise ratio ($\mathrm{SNR_q}$) of an ideal N-bit converter is defined by the formula $6.02 \cdot N + 1.76$. Using $N = 12$ for the DAC, this gives:

$$\mathrm{SNR_q} = 6.02 \cdot 12 + 1.76 \approx 74\,\mathrm{dB}. \tag{4.11}$$

This $74\,\mathrm{dB}$ theoretical limit provides comfortable headroom for a guitar signal [3].

4.6 Verification

Two checks were made on the assembled prototype: the pipeline latency against the threshold reported for live playing, and the noise floor at the output.

The pipeline latency from the input jack to the output jack is the $5.33\,\mathrm{ms}$ value derived above. This latency falls well below the $10\,\mathrm{ms}$ threshold typically cited as the onset of perceptible delay for live performance.

Using the recording chain described in Section 3, Audacity's Contrast tool returned an RMS level of $-19.91\,\mathrm{dB}$ for the $300\,\mathrm{Hz}$ reference tone and $-40.74\,\mathrm{dB}$ for the noise-only recording, giving

$$\mathrm{SNR}_{\mathrm{sys}} = -19.91 - (-40.74) = 20.83\,\mathrm{dB} \tag{4.12}$$

at the test signal level. This figure includes the amplifier's noise contribution, since the amplifier is part of the recorded chain. The tone signal did not clip in any stage of the recording. A frequency analysis of the same recording (Figure 11) shows the $300\,\mathrm{Hz}$ reference tone as a narrow spike rising above a broadband noise floor, with no narrow peak at $50\,\mathrm{Hz}$ or its harmonics; mains hum is therefore not a significant contributor. A low static was audible at the output during the test but did not mask the guitar tone or the echo when playing.

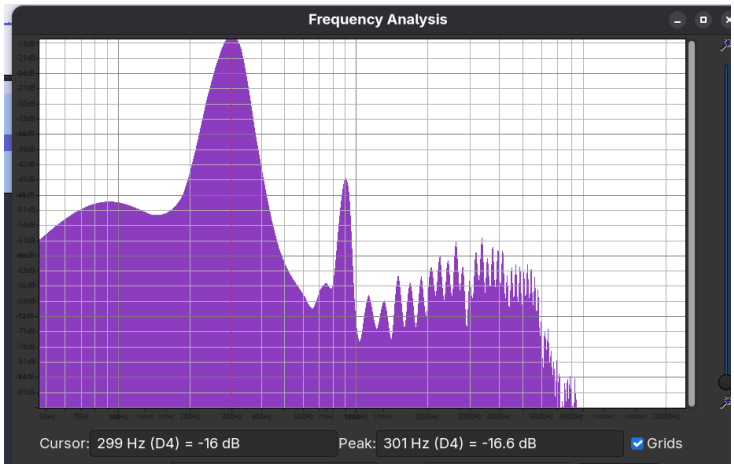

Figure 11: Frequency spectrum of the $300\,\mathrm{Hz}$ reference-tone recording at the amplifier USB output.

In summary, all subsystems: analog frontend, analog backend, firmware, and DSP algorithm functioned as specified, although audible noise was present.

5 Discussion

The project question was whether a working delay effect could be built around an STM32H743 and a single MCP6002 dual op-amp on a $3.3\,\mathrm{V}$ single supply, with an audible echo on real guitar input. The answer is yes. The frontend pushed the guitar signal into the range the ADC needed, the firmware ran the $300\,\mathrm{ms}$ delay with feedback at $48\,\mathrm{kHz}$ without dropouts, and the backend brought the result back to a level the amplifier accepts. On a real guitar the echo came through clearly. The problems that showed up during the work were all on the analog and assembly side, not in the firmware.

5.1 Limitations

A low static was audible at the output. It did not cover the guitar tone or the echo, but the system signal-to-noise ratio reported in Section 4.6 is only $20.83\,\mathrm{dB}$. There is a large difference between this result and the theoretical quantization-noise floor of $74\,\mathrm{dB}$ expected from a 12-bit DAC. This shows that the main noise comes from the analog circuit and the board layout. This gap happens for three main reasons:

- **Underutilized ADC Headroom:** The analog frontend is a unity-gain buffer. A standard passive guitar pickup generates a peak-to-peak signal of $100\,\mathrm{mV}$ to $300\,\mathrm{mV}$. Because we feed this unamplified signal into the ADC's full $3.3\,\mathrm{V}$ range, it uses less than 10% of the digital headroom. This wastes a lot of the digital resolution during sampling. It raises the noise floor before the signal reaches the DSP.

- **Shared Power Supply and Grounding:** The circuit was built on a hand-soldered protoboard with no continuous ground plane. The $3.3\,\mathrm{V}$ analog power comes directly from the development board, so it shares the line with

the fast ARM microcontroller. Digital current spikes on this shared power line change the DC bias voltage of the analog stages. This injects switching noise directly into the audio path.

- **Component Selection and Filtering:** The MCP6002 works well for $3.3\,\mathrm{V}$ single-supply circuits, but its input noise ($\approx\ 8\,\mathrm{nV}/\sqrt{\mathrm{Hz}}$) is too high for good audio. Also, the first-order passive RC anti-aliasing filter has a weak $-6\,\mathrm{dB}$/octave roll-off. At the $24\,\mathrm{kHz}$ Nyquist limit, the attenuation is only about $3.6\,\mathrm{dB}$. This lets high-frequency noise from the shared power supply alias back into the audible band.

PCB Reliability. The hand-soldered protoboard was physically fragile and caused bad connections during testing. Moving to a custom printed circuit board would fix these mechanical problems and also provide the ground planes needed to reduce the noise.

5.2 Future work

A custom two-layer PCB with a continuous ground plane and a separate analog $3.3\,\mathrm{V}$ regulator would address both the noise and the contact-reliability limitations identified above. Replacing the single-pole RC anti-aliasing and reconstruction filters with a second-order Sallen–Key topology would give a steeper roll-off above the audible band and reduce out-of-band noise reaching the converters.

The current enclosure is larger than a commercial pedal because the WeAct Mini development board takes up most of the floor area. Mounting the STM32H743 directly on a custom PCB would shrink the unit to a standard guitar-pedal size.

The prototype has no true-bypass switch, so the signal path is broken when the pedal is unpowered. A bypass relay or a mechanical true-bypass switch would be needed for stage use. On the firmware side, the current build runs a single fixed

effect; the H743 has enough flash and RAM to hold several DSP modules and switch between them at runtime, so adding a footswitch-selectable set of effects (chorus, reverb, distortion) would be a natural extension.

6 Glossary

ADC Analog-to-Digital Converter. 2, 3, 5

DAC Digital-to-Analog Converter. 3, 5

DMA Direct Memory Access, a peripheral that moves data between memory and another peripheral without CPU involvement. 4, 5, 12, 14

KiCad Open-source suite for schematic capture and printed circuit board design. 5

MCP6002 Dual rail-to-rail single-supply op-amp used in both the analog frontend and the analog backend. 2, 4, 5, 20

STM32CubeMX Graphical tool from ST that generates peripheral initialization code for STM32 microcontrollers. 5, 6

TIM6 One of the basic timers in the STM32H7, used here to trigger both the ADC and the DAC at $48\,\mathrm{kHz}$. 12, 13

7 Bibliography

References

[1] STMicroelectronics, "STM32H743 datasheet," [Online]. Available: https://www.st.com/resource/en/datasheet/stm32h743vi.pdf.

[2] Microchip Technology, "MCP6001/2/4 datasheet," [Online]. Available: https://www.microchip.com/en-us/product/MCP6002.

[3] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, https://www.dspguide.com/pdfbook.htm California Technical Publishing, 1997.

[4] Phil's Lab, "STM32 Audio Series," [Online]. Available: https://www.youtube.com/@PhilsLab.

[5] STMicroelectronics, "RM0433: STM32H742, STM32H743/753 and STM32H750 Value line MCUs reference manual," [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0433-stm32h742-stm32h743753-and-stm32h750-value-line-advanced-armbased-32bit-mcus-stmicroelectronics.pdf.